

LAB 04 Python

*Zakład Mechaniki Budowli i Zastosowań Informatyki
Wydział Inżynierii Lądowej
Politechnika Warszawska*

Koordynator przedmiotu: dr hab inż. Sławomir Czarnecki

Podstawy języka Python. Omówienie czwartej partii informacji z gramatyki języka Python.

Omówienie instrukcji pętli while/else oraz for/else.

Zad.1. Oblicz, dla dowolnie przyjętej wartości $n \in N$, dwie sumy:

$$S_n = 1 + 3 + 5 + \dots + 2n - 1 \quad , \quad Z_n = 2 + 4 + 8 + \dots + 2^n$$

tj. sumy

$$S_n = \sum_{i=1}^n s_i \quad , \quad Z_n = \sum_{i=1}^n z_i$$

gdzie

$$s_i = 2 \cdot i - 1 \quad , \quad z_i = 2^i \quad (i = 1, 2, \dots, n)$$

implementując pętlę while. Porównaj numeryczne wyniki ze znanymi wartościami tych sum

$$S_n = n^2 \quad , \quad Z_n = 2^{n+1} - 2 \quad .$$

Kod zadania będzie pokazany w czasie laboratorium, jako ilustracja implementacji pętli while.

Zad.2. Wykorzystując warunek konieczny $\lim_{i \rightarrow \infty} a_i = 0$ zbieżności nieskończonego szeregu

$\sum_{i=0}^{\infty} a_i < \infty$, **zaimplementuj obliczanie przybliżonej wartości sumy szeregu geometrycznego**

$\sum_{i=0}^{\infty} a_i = a_0 + a_1 + a_2 + \dots = 1 + q + q^2 + \dots = \frac{1}{1-q}$, gdzie $|q| < 1$. **Obliczanie sumy szeregu ma być**

przerwane wtedy kiedy wartość bezwzględna $|a_i|$ będzie mniejsza lub równa zadanej liczbie $eps > 0$. Przetestuj kod dla $q = -0.3$ i $q = 0.9$ oraz $eps = 1.0 \cdot 10^{-10}$.

Kod zadania będzie pokazany w czasie laboratorium, jako ilustracja implementacji pętli while.

Zad.3. Zaimplementuj Algorytm Euklidesa (używając pętli *while*) znajdowania Największego Wspólnego Dzielnika (*NWD*) liczb naturalnych $a, b \in \mathbb{N}$

Krok 1. **dopóty dopóki $a \neq b$ wykonuj:**

jeśli $a > b$ to zmniejsz a o b , tzn.:

$$a = a - b$$

w przeciwnym przypadku zmniejsz b o a , tzn.:

$$b = b - a$$

Krok 2. $NWD = a$

Wczytanie liczb a i b dowolne, wynik *NWD* wyświetl na ekranie konsoli. Wynik powinien zostać wyświetlony na ekranie konsoli np. w następującej postaci:

$$NWD(80, 120) = 40$$

Zad. 4. Zaimplementuj, wykorzystując między innymi pętlę *while*, algorytm sprawdzenia czy liczba naturalna $n > 1$ jest liczbą pierwszą. Wprowadź w kodzie zmienną o nazwie *pierwsza*, która ma być ostatecznie równa *True* lub *False* w zależności od tego czy liczba n okaże się być liczbą pierwszą czy też nie.

Algorytm sprawdzania czy n jest liczbą pierwszą

- Jeśli $n = 2$ lub $n = 3$, to stwierdzamy, że n **jest** liczbą pierwszą,
- w przeciwnym przypadku, jeśli liczba 2 lub liczba 3 dzieli n , to stwierdzamy, że n **nie jest** liczbą pierwszą,
- w przeciwnym przypadku, sprawdzać będziemy czy n jest podzielne przez i lub przez $i + 2$, dla $i = 5, 11, 17, 23, 29, \dots$ takiego, że $i^2 \leq n$. Jeśli okazałoby się, że i lub $i + 2$ **dzieli** n , to stwierdzamy, że n **nie jest** liczbą pierwszą, przy czym wtedy bezwzględnie **przerywamy** dalsze sprawdzenia wychodząc natychmiast z pętli *while* **wykorzystując instrukcję *break***. Jeśli żadna z liczb i lub $i + 2$, dla $i = 5, 11, 17, 23, 29, \dots$ takiego, że $i^2 \leq n$ nie dzieli n , to stwierdzamy, że n **jest** liczbą pierwszą.

Powyższy algorytm sprawdzenia czy n jest liczbą pierwszą wynika z następującego lematu:

Lemat. Jeżeli liczby 2, 3 oraz 5, 7, 11, 13, $\dots$, $6 \cdot i - 1$, $6 \cdot i + 1$, $\dots$ ($i = 1, 2, \dots$) **nie są dzielnikami** n , to n **jest liczbą pierwszą**. Przetestuj swój kod dla następujących liczb n :

$$n0 = 5 \text{ (pierwsza = True),}$$

$$n4 = 1234547 \text{ (pierwsza = True),}$$

$$n1 = 2147483647 \text{ (pierwsza = True),}$$

$$n5 = 104727 \text{ (pierwsza = False),}$$

$$n2 = 74747474747474747 \text{ (pierwsza = True),}$$

$$n6 = 104729 \text{ (pierwsza = True).}$$

$$n3 = 12345678987654321 \text{ (pierwsza = False),}$$

Informację o tym czy liczba n jest pierwsza czy też lub nie jest pierwsza **wyświetl na ekranie konsoli** wykorzystując do tego celu, między innymi wartość zmiennej *pierwsza* oraz wyrażenie trójargumentowe $A = Y \text{ if } X \text{ else } Z$, z funkcją *print(...)*. Wynik powinien zostać wyświetlony na ekranie konsoli np. w następującej postaci:

LICZBA n = 104729 jest liczbą pierwszą

Zad. 5. Zaimplementuj, następujący algorytm: ustal wartość zmiennej *K* oraz wartość maksymalnej liczby prób w zmiennej *MAKSYMALNA_LICZBA_PROB* inicjalizując je dowolnymi liczbami naturalnymi, na przykład odpowiednio **10** oraz **20**. W pętli *while* losuj liczby całkowite z przedziału $[-K, K]$ wyświetlając je kolejno na ekranie. W przypadku gdyby wylosowana została liczba 0, tylko przerwij iterację, natomiast w przypadku kiedy kolejne wylosowane liczby nie będą równe 0, kontynuuj iterację, nie przekraczając maksymalnej dopuszczalnej ich liczby zdefiniowanej przez wartość zmiennej *MAKSYMALNA_LICZBA_PROB*. W przypadku, gdyby udało się w czasie wszystkich iteracji NIE WYLOSOWAĆ zera, program ma dodatkowo wyświetlić następującą informację:

```
-10 7 10 9 2 -7 2 -4 2 -3 10 1 2 6 7 -2 -2 -10 -9 8 PRZY LICZBIE PROB = 20 OSIAGNELISMY SUKCES, PONIEWAZ UDAŁO SIĘ NAM PRZEŻYĆ NIE LOSUJĄC ZABÓJCZEGO ZERA
```

i tylko taką informację ma co najwyżej dodatkowo wyświetlić.

Kod zadania będzie pokazany na laboratorium, jako przykład implementacji pętli *while* z użyciem instrukcji *break* oraz *continue* w języku Python jako niezdefiniowanego w ten sposób odpowiednika instrukcji pętli *while* w innych językach programowania.

Zad. 6. Przykłady implementacji pętli *for* dla kilku różnych obiektów reprezentujących uporządkowane sekwencje:

```
['Wish', 'You', 'Were', 'Here']
['Wish You Were Here']
'Wish You Were Here'
[('Wish', 'You'), ('', ''), ('Were', 'Here')]
```

Kody przykładów będą zademonstrowane na laboratorium.

Zad. 7. Kilka przykładów implementacji pętli *for* po jednym obiekcie [1, 2, 3, 4] i po dwóch obiektach razem:

```
'ALICJA Z KRAINY ZAPOMNIENIA'
'ODKRYCIA SWOJEGO OCZEKUJE'
```

w celu utworzenia listy zawierającej wspólne znaki w obu zadanych łańcuchach znaków z jednoczesnym zadbanie o to, aby wspólne znaki w tej nowo tworzonej liście się nie powtarzały. W jednym wariancie użyte będą

dwie pętle *for* (z *in*) oraz jedna instrukcja warunkowa *if*,

a w drugim wariancie (typowym dla Pythona, ale nietypowym dla innych języków) użyta będzie tylko

jedna pętla *for* (z *in*) wraz z jedną instrukcją warunkową *if*, ale dodatkowo z *in*, co znacząco spowoduje zmniejszenie się liczby wszystkich iteracji.

Kody przykładów będą zademonstrowane na laboratorium.

Zad. 8. Zastosowanie funkcji *range* w pętli *for* jako podstawowego iteratora, najczęściej wykorzystywanego do generowania indeksów dla pętli *for*, zwłaszcza wtedy kiedy niezbędna jest lista liczb całkowitych o zdefiniowanym zakresie. Zwrócenie uwagi na bardzo ważny fakt, że iteracja po generowanych przez *range* indeksach identyfikujących elementy obiektu, np.:

```
for element in range(0, len(obiekt), co_ile)
```

jest z reguły wolniejsza od iteracji przebiegającej bezpośrednio po elementach tego obiektu

```
for element in obiekt[::co_ile].
```

Uwagi dotyczące modyfikacji obiektów będących listami, kiedy konieczne jest użycie indeksów przez odwołanie się do iteratora *range* w przypadku kiedy np. modyfikujemy elementy listy w pętli.

Użycie kilku innych ważnych funkcji takich jak: *append*, *lower*, *upper*.

Tworzenie tzw. *list składanych* w zastosowaniu do pisania kodu w bardzo efektywny i jednocześnie nietypowy, dla tradycyjnych języków programowania, sposób.

Tworzenie *list składanych* za pomocą wartości generowanych przez zadaną formułę na tych elementach pewnego obiektu, które spełniają jednocześnie warunek zdefiniowany przez inną zadaną formułę.

Kody przykładów będą zademonstrowane na laboratorium.

Zad. 9. Zaimplementuj obliczanie sum szeregów S_n, Z_n z **Zad. 1** przy użyciu pętli *for* zamiast *while*.

Poniżej wydruk przewidywanego do napisania w języku Python kodu w trakcie laboratorium z Informatyki I. W przypadku zadań przewidzianych do samodzielnego zrobienia, zaleca się korzystanie z tych kodów, jednak dopiero po własnych próbach (udanych lub nie) sklecenia własnych ich wersji – głównie w celu porównania z sugerowanymi rozwiązaniami, spośród których, poprawnych ich wariantów, może być z reguły bardzo wiele.

```
import math
import random
#PĘTLE - omówić dokładnie pętle w Pythonie
#while <wyrażenie w warunku ma wartość logiczną True>:
#   <instrukcje w ciele pętli>
#else:
#   <opcjonalny wariant: instrukcje, ale wykonane tylko wtedy jeśli wyjście z pętli
nie było spowodowane przez instrukcję break>

#while <test_0>:
#   <instrukcje w ciele pętli>
#   if <test_1>
#       break          #WYJŚCIE Z PETLI, POMINIĘCIE RESZTY
#   if <test_2>
#       continue      #PRZEJSCIE DO SPRAWDZENIA test_0
#else:
#   <opcjonalny wariant: instrukcje, ale wykonane tylko wtedy jeśli wyjście z pętli
nie było spowodowane przez instrukcję break>

#Zad.1
n = 10
Sn = 0
Zn = 0
i = 1
while i <= n:
    si = 2 * i - 1
    zi = 2 ** i
    Sn += si
    Zn += zi
    i += 1
print('Sn = ', Sn, ' | Analitycznie Sn = ', n ** 2)
print('Zn = ', Zn, ' | Analitycznie Zn = ', 2 ** (n+1) - 2)
print()
```

```

#Zad.2
#eps = float(input('Wczytaj dodatnią liczbę > 0: eps = '))
#q = float(input('Wczytaj liczbę q taką, że |q|<1, q = '))
eps = 1.0e-6
q = -0.9
ai = 1
S = ai
while math.fabs(ai) > eps:
    ai *= q
    S += ai
print('S = ', S, ' | Analitycznie S = ', 1 / (1 - q))
print()

#Zad.3
#a = int(input('Wczytaj dodatnią liczbę całkowitą a = '))
#b = int(input('Wczytaj dodatnią liczbę całkowitą b = '))
A = a = 80
B = b = 120
while a != b:
    if a > b:
        a -= b
    else:
        b -= a
print('NWD(' , A , ', ' , B , ') = ', a)

#Zad.4
'''
n0 = 5 TAK
n1 = 2147483647 TAK
n2 = 747474747474747 TAK
n3 = 12345678987654321 NIE
n4 = 1234547 TAK
n5 = 104727 NIE
n6 = 104729 TAK
'''

#n = int(input('Wczytaj dodatnią liczbę całkowitą n = '))
n = 104729
if n == 1:
    pierwsza = False
elif n == 2 or n == 3:
    pierwsza = True
elif n % 2 == 0 or n % 3 == 0:
    pierwsza = False
else:
    i = 5
    pierwsza = True
    while i ** 2 <= n:
        if n % i == 0 or n % (i + 2) == 0:
            pierwsza = False
            break
        i += 6
INFO = ' jest liczbą pierwszą' if pierwsza else ' nie jest liczbą pierwszą'
print('LICZBA n = ', n, INFO)
print()
print()

```

```

#Zad.5
K = 10
LICZBA_PROB = 0
MAKSYMALNA_LICZBA_PROB = 20
while LICZBA_PROB < MAKSYMALNA_LICZBA_PROB:
    r = random.randint(-K , K)
    print(r, end=' ')
    if r == 0:
        break
    else:
        LICZBA_PROB += 1
        continue
else:
    print('PRZY LICZBIE PROB = ',LICZBA_PROB, ' OSIAGNELISMY SUKCES, PONIEWAZ UDAŁO
SIĘ NAM PRZEŻYĆ NIE LOSUJĄC ZABÓJCZEGO ZERA')
print('\n')

#Pętla for jest szybsza od while, bo for zalicza się do pętli w stylu liczników
#W pętli for(...) jesteśmy w stanie identyfikować wszystkie elementy należące do
dowolnego obiektu będącego najczęściej uporządkowaną
sekwencją, tj. łańcuchem znaków, listą, krotką i/lub wieloma innymi jeszcze obiektami
(nawet niesekwencyjnymi !), po których można
iterować.
#Wykonanie pętli polega z reguły i najczęściej na wykorzystywaniu kolejnych elementów
obektu sekwencyjnego, w celu wykonania
zadanych działań zależnych od każdego z tych elementów.

#for <cel> in <obekt>: #Przypisywanie elementów obiektów do celu będącego z reguły
zmienną identyfikującą elementy obiektu sekwencyjnego
#   <instrukcje w ciele pętli>
#else:
#   <opcjonalny wariant: instrukcje, ale wykonane tylko wtedy jeśli wyjście z pętli
nie było spowodowane przez instrukcję break>

#for <cel> in <obekt>: #Przypisanie elementów obiektów do celu
#   <instrukcje w ciele pętli>
#   if <test_1>
#       break          #WYJŚCIE Z PETLI, POMINIĘCIE RESZTY
#   if <test_2>
#       continue      #PRZEJSCIE DO SPRAWDZENIA test_0
#else:
#   <opcjonalny wariant: instrukcje, ale wykonane tylko wtedy jeśli wyjście z pętli
nie było spowodowane przez instrukcję break>

#Zad. 6
#Wyświetlenie zawartości listy
liczba_iteracji = 0
obekt = ['Wish', 'You', 'Were', 'Here']
for cel in obiekt:          #Pętla for... po 4 elementach listy, z których każdy jest
łańcuchem (string)
    print(cel, end=' ')     #a w każdej iteracji wyświetlany jest kolejny element -
string
    liczba_iteracji += 1
print()
print('liczba iteracji = ', liczba_iteracji)
print()

```

```

#Wyświetlenie zawartości listy
liczba_iteracji = 0
obiekt = ['Wish You Were Here']
for cel in obiekt:          #Pętla for... po 1 elemencie listy, którym jest łańcuch
znaków (string)
    print(cel, end=' ')      #a w każdej iteracji wyświetlany jest kolejny, tutaj tylko
raz i tylko ten jeden element - string
    liczba_iteracji += 1
print()
print('liczba_iteracji = ', liczba_iteracji)
print()

```

```

#Wyświetlenie zawartości listy
liczba_iteracji = 0
obiekt = 'Wish You Were Here'
for cel in obiekt:          #Pętla for... po 18 elementach listy, z których każdy jest
pojedynczym znakiem (w tym także trzy są znakiem spacji)
    print(cel, end='')      #a w każdej iteracji wyświetlany jest kolejny element -
pojedynczy znak
    liczba_iteracji += 1
print()
print('liczba_iteracji = ', liczba_iteracji)
print()

```

```

#Wyświetlenie zawartości listy
liczba_iteracji = 0
obiekt = [('Wish', 'You') , ('', ''), ('Were' , 'Here')]
for cel0 , cel1 in obiekt:  #Pętla for... po 3 elementowej liście krotek i
jnocześnie po obu elementach każdej krotki
                                #zawierającej dwa łańcuchy znaków co sprawia,
    print(cel0, cel1, end='')  #że w każdej iteracji wyświetlane są elegancko
łańcuchy znaków w krotkach, a nie krotki łańcuchów znaków
    liczba_iteracji += 1
print()
print('liczba_iteracji = ', liczba_iteracji)
print()

```

```

#Wyświetlenie zawartości listy
liczba_iteracji = 0
obiekt = [('Wish', 'You') , ('', ''), ('Were' , 'Here')]
for cel in obiekt:          #Tutaj podobnie jak w poprzednim kodzie mamy pętlę
for... po 3 elementowej liście krotek, jednak tutaj
                                #w każdej iteracji wyświetlane są krotki łańcuchów
znaków - czyli jest to wizualnie chyba gorszy wariant
    print(cel, end='')
    liczba_iteracji += 1
print()
print('liczba_iteracji = ', liczba_iteracji)
print()

```

#Zad. 7

```

#Utworzenie listy zawierającej wspólne znaki w obu zadanych łańcuchach znaków z
jednoczesnym zadbanie o to,
#aby wspólne znaki się nie powtarzały.
#Poniżej stosujemy dwie pętle, po dwóch różnych obiektach z warunkiem na dopisywanie
niepowtarzających się ich wspólnych elementów
#(wykorzystanie operatora == oraz metody count(element) zliczającej zadany element w
obiekcie typu lista).
#Mamy tu dwie pętle "for" z "in" (co jest regułą) oraz sam "if"
#Dodatkowo sortujemy (w miejscu) finalną listę wspólnych elementów obu tych obiektów
za pomocą metody sort().

```

```

liczba_iteracji = 0
obiekt0 = 'ALICJA Z KRAINY ZAPOMNIENIA'
obiekt1 = 'ODKRYCIA SWOJEGO OCZEKUJE'
product = []
for cel0 in obiekt0:
    for cel1 in obiekt1:
        if cel0 == cel1 and product.count(cel0) < 1:
            product.append(cel0)
            liczba_iteracji += 1
print()
product.sort()
print('product', product)
print('liczba_iteracji = ', liczba_iteracji)
print()

```

#Poniżej stosujemy nie dwie, ale tylko jedną pętlę, ale z nieco zmodyfikowanym warunkiem na dopisywanie niepowtarzających się ich wspólnych elementów, w którym mamy niejawnie sprawdzanie jednoczesnego istnienia oraz powtarzalności elementu z pierwszego obiektu w drugim obiekcie. #Mamy tu jedną pętlę "for" z "in" (co jest regułą) oraz jeden "if" z "in" (co jest alternatywą dla instrukcji "if")

```

liczba_iteracji = 0
obiekt0 = 'ALICJA Z KRAINY ZAPOMNIENIA'
obiekt1 = 'ODKRYCIA SWOJEGO OCZEKUJE'
product = []
for cel0 in obiekt0:
    #operator in poniżej w niejawnym sposób przeszukuje obiekt, szukając dopasowania (przynajmniej logicznego)
    #co pozwala na zastąpienie pętli for cel1 in obiekt1: instrukcją if cel0 in obiekt1
    if cel0 in obiekt1 and product.count(cel0) < 1:
        product.append(cel0)
        liczba_iteracji += 1
print()
product.sort()
print('product', product)
print('liczba_iteracji = ', liczba_iteracji)
print()

```

```

liczba_iteracji = 0
obiekt = [1, 2, 3, 4]
sum = 0
for cel in obiekt:
    sum += cel
    liczba_iteracji += 1
print('suma = ', sum)
print('liczba_iteracji = ', liczba_iteracji)

```

#Funkcja range jest narzędziem, który można zastosować w bardzo wielu kontekstach, ale najczęściej wykorzystywana jest do generowania indeksów dla pętli for, kiedy potrzebna jest lista liczb całkowitych. W Pythonie, range jest iteratorem, więc jeśli chcemy zobaczyć co tak na prawdę jest przez niego generowane, musimy wywołanie tej funkcji "opakować" jawnym rzutowaniem na listę, co z reguły nie jest pożądane, aby to jawnie generować (chyba, że na wstępnym etapie sprawdzania poprawności kodu).


```

#Niech start, finish, co_ile będą liczbami całkowitymi

#Zad. 8
print()

start = -10
print(list(range(start)))          #[]
print()

start = 10
print(list(range(start)))          #[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print()

start = -10
finish = 20
print(list(range(start, finish)))
#[-10, -9, -8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,
13, 14, 15, 16, 17, 18, 19]
print()

start = 10
finish = 20
print(list(range(start, finish)))  #[10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
print()

start = -10
finish = 20
co_ile = 2
print(list(range(start, finish, co_ile)))  #[-10, -8, -6, -4, -2, 0, 2, 4, 6, 8, 10,
12, 14, 16, 18]
print()

start = 10
finish = 20
co_ile = 2
print(list(range(start, finish, co_ile)))  #[10, 12, 14, 16, 18]
print()

obiekt = ['ALICJA' , 'Z' , 'KRAINY' , 'ZAPOMNIENIA' , 'ODKRYCIA' , 'SWOJEGO' ,
'OCZEKUJE']
print(obiekt)
wynik = []
co_ile = 3
for cel in range(0, len(obiekt), co_ile):
    wynik.append(obiekt[cel])
print(wynik)          #['ALICJA', 'ZAPOMNIENIA', 'OCZEKUJE']

#TO PONIZEJ JEST SZYBSZE !!!
obiekt = ['ALICJA' , 'Z' , 'KRAINY' , 'ZAPOMNIENIA' , 'ODKRYCIA' , 'SWOJEGO' ,
'OCZEKUJE']
print(obiekt)
wynik = []
co_ile = 3
for cel in obiekt[::co_ile]:
    wynik.append(cel)
print(wynik)          #['ALICJA', 'ZAPOMNIENIA', 'OCZEKUJE']

#PROBA MODYFIKACJI LISTY
obiekt = [0 , 1 , 2 , 3 , 4 , 5 , 6]
print(obiekt)
for cel in obiekt:

```

```

    cel = -cel      #TAKA MODYFIKACJA LISTY obiekt NIE ZADZIAŁA POMIMO, ŻE TUTAJ cel
JEST obiektem
print(obiekt)      #[0, 1, 2, 3, 4, 5, 6]
#bo aby móc zmodyfikować listę w pętli po jej elementach, musimy użyć indeksów

obiekt = [0 , 1 , 2 , 3 , 4 , 5 , 6]
print(obiekt)
for cel in range(len(obiekt)):
    obiekt[cel] = -obiekt[cel]      #TAKA MODYFIKAC JEST KONIECZNA, ALE cel JEST TUTAJ
OBIEKTEM TYPU int, A NIE TYPU obiekt
print(obiekt)      #[0, -1, -2, -3, -4, -5, -6]

#Znacznie bardziej przekonujący przykład poniżej !!!
obiekt = ['A' , 'B' , 'C' , 'D' , 'E' , 'F' , 'G']
print(obiekt)
for cel in obiekt:
    cel = cel.lower()               #TAKA MODYFIKACJA LISTY obiekt NIE
ZADZIAŁA POMIMO, ŻE TUTAJ cel JEST obiektem
print(obiekt)                       #['A', 'B', 'C', 'D', 'E', 'F', 'G']

obiekt = ['A' , 'B' , 'C' , 'D' , 'E' , 'F' , 'G']
print(obiekt)
for cel in range(len(obiekt)):
    obiekt[cel] = (obiekt[cel]).lower()      #TAKA MODYFIKAC JEST KONIECZNA, JEDNAK cel
JEST TUTAJ OBIEKTEM TYPU int, A NIE TYPU obiekt
print(obiekt)                              #['a', 'b', 'c', 'd', 'e', 'f', 'g']

lista0 = [ 1,  2,  3,  4,  5,  6,  7,  8,  9]
lista1 = [-9, -8, -7, -6, -5, -4, -3, -2, -1]
lista2 = [-1, -1, -1, -1, -1, -1, -1, -1, -1]
print('lista lista0:', lista0)
print('lista lista1:', lista1)
print('lista lista2:', lista2)
print('lista list(zip(lista0, lista1)):', list(zip(lista0, lista1, lista2)))
#[(1, -9, -1), (2, -8, -1), (3, -7, -1), (4, -6, -1), (5, -5, -1), (6, -4, -1), (7, -
3, -1), (8, -2, -1), (9, -1, -1)]

#Zadanie polega na zsumowaniu element po elemencie oraz wymnożeniu element po
elemencie wszystkich kolejnych składowych tych trzech
#powyższych list
#Sumę i iloczyn tych trzech list definiujemy zatem następująco:
#sumą jest lista      [1   -9   -1 , 2   -8   -1 , ..., 9   -1   -1 ] = [-9,
-7, ..., 7]
#iloczynem jest lista [1 x (-9) x (-1), 2 x (-8) x (-1), ..., 9 x (-1) x (-1)] = [ 9,
16, ..., 9]
#Tradycyjny sposób realizacji tego zadania, typowy dla wielu innych klasycznych
języków programowania
suma_list = []
iloczyn_list = []
for i in range(len(lista0)):
    suma_list.append (lista0[i] + lista1[i] + lista2[i])
    iloczyn_list.append(lista0[i] * lista1[i] * lista2[i])
print('suma_list_element_po_elemencie = ', suma_list)
print('iloczyn_list_element_po_elemencie = ', iloczyn_list)

```

```

#To samo zadanie, można zaimplementować zupełnie inaczej w Pythonie z kodem, który
znacząco przyspieszy obliczenia numeryczne
suma_list = []
iloczyn_list = []
#Wykorzystując iterator zip(...) dla trzech list, można zaimplementować powyżej
zdefiniowane dodawanie oraz mnożenie
#w ramach przeprowadzanej jednocześnie po trzech listach iteracji trójelementowych
krotek, powstałych po "zzipowaniu".
#Iteracja przeprowadzana będzie w tej nowej wersji bezpośrednio po elementach trzech
list, NIE odwołując się przy tym do indeksów
#(tworzonych w poprzedniej klasycznej wersji poprzez range(...)) identyfikujących
składowe tychże list, co okazuje się być sposobem
#implementacji pętli spowalniającym w Pythonie jej wykonanie w porównaniu ze sposobem
bezpośredniego odwoływania się do elementów
#tych list.
for (a, b, c) in zip(lista0, lista1, lista2):
    suma_list.append(a + b + c)
    iloczyn_list.append(a * b * c)
    print('dodawanie list element po elemencie: ', a, b, c, a + b + c)
    print('mnożenie list element po elemencie: ', a, b, c, a * b * c)
print('suma_list_element_po_elemencie = ', suma_list)
print('iloczyn_list_element_po_elemencie = ', iloczyn_list)
#suma_list_element_po_elemencie = [-9, -7, -5, -3, -1, 1, 3, 5, 7]
#iloczyn_list_element_po_elemencie = [ 9, 16, 21, 24, 25, 24, 21, 16, 9]

#Konstrukcja listy składanej jest zapisywana w nawiasach kwadratowych. Jest to
opcjonalna metoda tworzenia nowej listy.
#Kod list składanych działa o wiele szybciej niż kod implementujący jawnie petle for
(nie wspominając o while),
#ponieważ iteracje są wykonywane wewnątrz interpretera z szybkością języka C, a nie z
szybkością kodu Pythona.
#Dlatego warto jest je stosować, choć jest to oczywiście tylko opcjonalny wariant
implementacji definiowania list.
#Wyrażenie:
#lista = [f(i) for i in X if g(i)]
#gdzie: X jest obiektem pewnego typu sekwencyjnego, tworzy listę z wartości zwracanych
przez funkcję f (lub przez jakieś wyrażenie)
#na tych elementach obiektu X, które spełniają warunek zwracany przez funkcję g (lub
przez jakieś wyrażenie).

#Poniżej przykład utworzenia listy składanej, która zawiera kolejno kwadraty
parzystych liczb naturalnych od 0 do 9
lista = [i ** 2 for i in range(10) if i % 2 == 0]
print(lista)
# [0, 4, 16, 36, 64]

#Poniżej przykład utworzenia listy krotek
...
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (1, 8), (1, 9)]
[(2, 1), (2, 2), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9)]
[(3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6), (3, 7), (3, 8), (3, 9)]
[(4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (4, 8), (4, 9)]
[(5, 1), (5, 2), (5, 3), (5, 4), (5, 5), (5, 6), (5, 7), (5, 8), (5, 9)]
[(6, 1), (6, 2), (6, 3), (6, 4), (6, 5), (6, 6), (6, 7), (6, 8), (6, 9)]
[(7, 1), (7, 2), (7, 3), (7, 4), (7, 5), (7, 6), (7, 7), (7, 8), (7, 9)]
[(8, 1), (8, 2), (8, 3), (8, 4), (8, 5), (8, 6), (8, 7), (8, 8), (8, 9)]
[(9, 1), (9, 2), (9, 3), (9, 4), (9, 5), (9, 6), (9, 7), (9, 8), (9, 9)]
...

M = 10 #Liczba wierszy
N = 10 #Liczba kolumn
macierz_krotek = [[(i, j) for j in range(1, N)] for i in range(1, M)]
for row in macierz_krotek:
    print(row)

```

```
#Zad.9
print()
n = 10
Sn = 0
Zn = 0
for i in range(1, n + 1):
    Sn += 2 * i - 1
    Zn += 2 ** i
print('Sn = ', Sn, ' | Analitycznie Sn = ', n ** 2)
print('Zn = ', Zn, ' | Analitycznie Zn = ', 2 ** (n+1) - 2)
print()
```